

bikiDATA: A Python Library to Query and Explore Large-Scale RDF Datasets

<https://arxiv.org/abs/2608.20358>

1. Problem

- 문제: 대규모 RDF/Knowledge Graph를 Python 애플리케이션에서 활용하려면 SPARQL과 복잡한 데이터 매핑을 알아야 하는 높은 진입장벽이 존재.
- 왜 중요한가: Wikidata 같은 대규모 RDF 데이터는 기존 public SPARQL endpoint의 timeout·결과 제한 때문에 실제 애플리케이션에서 직접 활용하기 어렵다.

2. Method

- 핵심 아이디어: RDF를 Python에서 native하게 조회·분석할 수 있는 라이브러리 **bikiDATA** 제안.
- 기존 방법과 차이:
 - SPARQL을 직접 작성하는 대신 Python API로 RDF query 수행.
 - 조회 결과를 Python 구조로 바로 반환.
 - Full-text search, semantic/embedding search, visual similarity search까지 하나의 인터페이스에서 지원.
- 핵심 구성: DuckDB 기반 저장/쿼리 엔진 + RDF를 relation table 형태로 변환 + IRI/literal hash를 활용한 대규모 데이터 처리.
- SPARQL을 완전히 대체하는 것이 아니라, Python 중심의 새로운 RDF query 방식을 제공한다는 점이 핵심.

3. Experiments & Results

- 실험: Sparqloscope benchmark에서 3초 이상 걸리는 query를 대상으로 성능 측정.
- 환경: 500GB RAM / 96 cores 서버에서 QLever와 비교.
- 결과:

- cold-cache 조건에서 bikiDATA가 QLever보다 빠른 query 실행 성능을 보임.
- 대규모 RDF dataset에서도 실용적인 시간 내 query가 가능함을 보임.
- 단순 query 성능뿐 아니라 Python에서 다양한 검색 기능을 함께 사용할 수 있다는 개발 편의성도 개선.

4. Insight

- 핵심 기여 한 줄:
 - 대규모 RDF를 SPARQL 전문가가 아니어도 Python 데이터 분석처럼 다룰 수 있게 만든 RDF query/analysis layer.
- 한계점:
 - 모든 SPARQL 표현력을 지원하지 않음.
 - 성능 실험 범위가 상대적으로 제한적.
 - embedding/visual similarity 등 일부 기능은 아직 개발 단계.
- 내가 가져갈 포인트:
 - 핵심은 "SPARQL을 더 빠르게"가 아니라 "RDF를 Python 개발자가 쉽게 쓰게" 만드는 것.
 - 특히 DuckDB + RDF + embedding search 조합은 Knowledge Graph를 RAG/검색/데이터 분석 pipeline에 연결할 때 참고할 만한 방향.